Analyzing C/C++ Library Migrations at the Package-level: Prevalence, Domains, Targets and Rationales across Seven Package Management Tools

HAIQIAO GU and YILIANG ZHAO, School of Computer Science, Peking University, China and Key Laboratory of High Confidence Software Technologies, Ministry of Education, China
KAI GAO, School of Computer & Communication Engineering, University of Science and Technology Beijing, China
MINGHUI ZHOU, School of Computer Science, Peking University, China and Key Laboratory of High Confidence Software Technologies, Ministry of Education, China

Library migration, i.e., replacing a third-party library with another, happens when a library can not meet the project's requirements and is non-trivial to accomplish. To mitigate the problem, substantial efforts have been devoted to understanding its characteristics and recommending alternative libraries, especially for programming language (PL) ecosystems with a central package hosting platform, such as Python (PyPI), JavaScript (npm), and Java (Maven). However, to the best of our knowledge, understanding of C/C++ library migrations is still lacking, possibly due to challenges resulting from the fragmented and complicated dependency management practices in the C/C++ ecosystem. Given the widespread use and critical role of C/C++, the lack of such understanding hinders the formulation of best practices and tools to facilitate library migrations in the C/C++ ecosystem. To bridge this knowledge gap, this paper analyzes 124,786 dependency configuration file changes in 19,943 C/C++ projects that utilize different package management tools and establishes a C/C++ library migration dataset, comprising 2,191 migrations and 717 migration rules. Based on the dataset, we investigate the prevalence, domains, target library, and rationales of C/C++ library migrations and compare the results with three widely investigated PLs: Python, JavaScript, and Java. We find that the overall trend in the number of C/C++ library migrations is similar to Java, which grew slowly and stabilized, and then decreased slightly. Notably, the number of migrations in emerging package management tools, such as Meson and Vcpkg, is on the rise. Migrations across different package management tools are also observed, with the most common being from Git submodules to Conan. In C/C++, library migrations mainly occur in three domains, i.e., GUI, Build, and OS development, but are rare in domains (e.g., Testing and Logging) that dominate library migrations in the three compared PLs. 83.46% of C/C++ source libraries only have one migration target, suggesting that our library migration dataset could be used directly to recommend migration targets for C/C++ developers. We find four C/C++-specific migration reasons, such as less compile time and unification of dependency management, revealing the unique dependency management requirements in C/C++ projects. We believe our

This work is sponsored by the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM101) and the National Natural Science Foundation of China (No. 62332001 and No. 62502030).
Authors' address: H. Gu and Y.Zhao, School of Computer Science, Peking University, No. 5 Yiheyuan Road, Haidian District, Beijing, 100871, China and Key Laboratory of High Confidence Software Technologies, Ministry of Education, China; e-mails: {ghq,zyl7}@stu.pku.edu.cn; K. Gao, School of Computer & Communication Engineering, University of Science and Technology Beijing, No. 30 Xueyuan Road, Haidian District, Beijing, 100083, China; e-mail: kai.gao@ustb.edu.cn; M. Zhou (Corresponding author), School of Computer Science, Peking University, No. 5 Yiheyuan Road, Haidian District, Beijing, 100871, China and Key Laboratory of High Confidence Software Technologies, Ministry of Education, China; e-mail: zhmh@pku.edu.cn;

ACM XXXX-XXXX/2026/4-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

findings can help C/C++ developers make more informed library migration decisions and shed light on the design of C/C++ library migration tools.

CCS Concepts: • **Software and its engineering → Software evolution**; **Maintaining software**; • **Human-centered computing → Open source software**.

Additional Key Words and Phrases: Library migration, C and C++ ecosystem, mining software repositories

ACM Reference Format:
Haiqiao Gu, Yiliang Zhao, Kai Gao, and Minghui Zhou. 2026. Analyzing C/C++ Library Migrations at the Package-level: Prevalence, Domains, Targets and Rationales across Seven Package Management Tools. 1, 1 (April 2026), 24 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Current software projects commonly reuse third-party libraries to boost productivity and ensure code quality [1, 50, 56]. However, reusing third-party libraries is not a one-time solution. As the project and third-party libraries evolve, the reused libraries may not meet the project's functional, legal, or performance requirements [22], and may even introduce security vulnerabilities [32, 57]. When such issues arise, a commonly adopted solution by developers is to replace the reused library with another one featuring the same or similar functionality, referred to as *library migration*. For example, commit mo-xiaoming/pl0-jit#f7e83b migrated from *doctest* to *catch2* for better value-parameterized test [17].

Library migration is a time-consuming and effort-intensive task that involves target library selection and code adaptation [6, 7, 54]. On average, a developer needs to learn about 4 APIs to perform a migration [26]. In commit cloudbotirc/- cloudbot#f8243223, a migration from *twitter* to *tweepy* even involves up to 35 unique APIs and 17 API mappings [14]. To ease the task, substantial efforts have been devoted to understanding the characteristics of library migrations [7, 20, 22, 26, 28, 46, 47, 55] and recommending target libraries [23, 24, 37] and APIs [4, 6, 10, 13]. These studies primarily focus on programming language (PL) ecosystems with central package hosting platforms, such as Maven for Java, npm for JavaScript, and PyPI for Python, where dependent libraries can be easily and uniquely identified from unified and structural dependency configuration files.

However, similar topics are rarely explored in the C/C++ ecosystem, while C/C++ developers also encounter challenges with finding alternatives during migration [25, 41, 42] (below is an example from stackoverflow).

> *In our project now we using log4cxx, but those library don't develop some years, also we have some problems with it. Could you advise some library for logging in C++. Library must support multithread logging, system-log ... Also lib license must be very democracy ... Must support linux, windows.* [25]

Unlike other PL ecosystems, dependency management in the C/C++ ecosystem is notoriously fragmented and complex [36]. Developers often rely on a mix of *package management tools* (PMTs), such as Conan and Git submodules, along with custom build systems and manual configurations [45]. When migrations occur, they may need to maintain multiple dependency configuration files. And the huge number and differences among packages hosted on various package management platforms further pose challenges for developers seeking alternatives. Compiling C/C++ packages across various platforms also burdens them. These characteristics make C/C++ developers particularly conservative when introducing or modifying dependencies, which may result in significant differences in library migration practices compared to other PLs. Given the popularity and significant role of C/C++ in critical areas such as operating systems (OS), embedded systems, and high-performance computing [9], we argue that it is urgent to advance the understanding of library migrations in the C/C++ ecosystem, especially its unique characteristics compared to

those in other ecosystems. Such an understanding could provide developers, maintainers, and stakeholders with valuable information to make more informed decisions and design automated tools for C/C++ library migrations.

Therefore, we attempt to bridge the gap in this study by analyzing a large body of C/C++ migrations. Specifically, we retrieve 717 migration rules and 2,191 migration instances from 124,786 dependency configuration file changes across 19,943 C/C++ projects. Based on this dataset, we analyze the prevalence, domains, target library, and rationale of C/C++ library migrations and compare the results with three widely studied PLs, i.e., Python, JavaScript, and Java. In particular, we answer the following research questions (RQs):

- **RQ1:** *How common do C/C++ migrations occur?*
  Motivation: This question aims to understand the prevalence and trend of C/C++ migrations and the migration frequency across different package management tools, which can highlight the urgency of investigating C/C++ library migrations and facilitate library migrations for specific package management tools in a more targeted way.
  Findings: The number of C/C++ library migrations grew slowly, stabilized gradually, and then decreased slightly, which is similar to that in the Java ecosystem. We find that the number of library migrations in emerging tools such as Meson and Vcpkg undergoes rapid growth. We also observe that some developers choose to switch between different package management tools, with the most common PMT migration pattern being from Git submodule to Conan.
- **RQ2:** *In which domains do C/C++ migrations occur?*
  Motivation: Migration domains vary across different ecosystems [20]. This question aims to understand the most common domain for C/C++ in which migrations occur and reveal frequent domains that require more attention.
  Findings: Build and GUI dominate nearly half of C/C++ library migrations. Notably, there are relatively few C/C++ migrations in domains like logging and testing (utility libraries), which take the most library migrations of the three compared PL ecosystems.
- **RQ3:** *What is the distribution of C/C++ migration targets chosen by developers?*
  Motivation: The migration targets represent the migration target libraries and may reflect the technology trend or best practices that developers turn to [40, 49], and developers' different choices of targets, i.e. their distribution reflects the divergence of technical trends in the C/C++ ecosystem. This question attempts to characterize the migration targets in the C/C++ ecosystem, providing insights for effectively selecting migration alternatives.
  Findings: Source libraries in most (83.46%) C/C++ library migration rules have only one migration target. C/C++ library migrations exhibit considerably stronger one-to-one and unidirectional characteristics than those in other PLs, indicating that universal migration practices may have been formed in C/C++. This finding also suggests that mining more comprehensive and accurate migration rules may be the right direction to recommend migration targets.
- **RQ4:** *Why do developers conduct C/C++ migrations?*
  Motivation: This question aims to understand the rationale for migrations in C/C++ and to find developers' specific considerations for C/C++ migration.
  Findings: We uncover four C/C++ specific migration reasons, such as less compile time and unification of dependency management. The main cause of C/C++ migrations includes the functionality of the target library and integration with platforms/other dependencies.

Our findings provide unique insights into the evolution of package management tools, migration recommendations, and library selection, to guide and inform developers on effectively managing

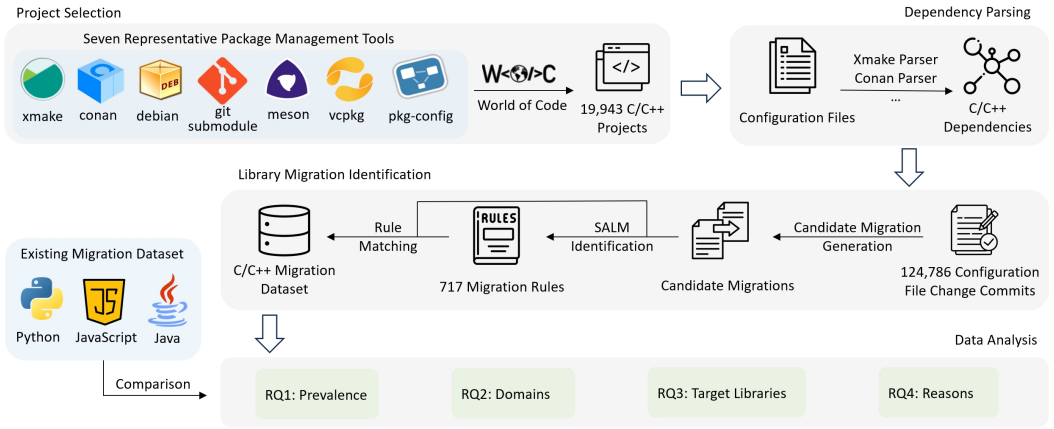

Fig. 1. Overview of our study

dependencies and migrations in C/C++ projects. We publicize our C/C++ migration dataset for further investigation.

2 Background and Related Work

Library migration refers to the process of replacing one library or dependency (source library, l_1) with another library (target library, l_2) that offers similar functionality [24, 29, 46, 47]. Usually, a library with another name means a different functionality group. However, for C/C++, the name of the library may always contain a version number, resulting in different names for different versions of the library (e.g., *libmutter-2*, *libmutter-3)*. We regard this kind of change as a library update rather than a migration. Notably, due to developers mixing multiple package management approaches in C/C++, a library may be removed from one package management tool while being added to another. In such cases, the modified package may have the same name. We also consider this a kind of library migration, i.e., PMT migration, which has not been discussed in previous studies. A migration rule is denoted as $\langle l_1, l_2 \rangle$ in previous studies [20, 22, 24, 47]. A PMT migration rule is denoted as $\langle pmt_1, pmt_2 \rangle$.

Migration is often necessary to address various issues, such as deprecated features [37], security vulnerabilities [22], performance improvements [3, 31], or compatibility requirements with other components of the software project [53]. It can also be driven by the need to take advantage of new features [31], better support [7], or licensing considerations [53] provided by alternative libraries [20, 22].

Empirical studies have been conducted to understand the nature of library migrations. Teyton et al. [46, 47] mined 1,198 migrations from 15,168 Java projects and summarized eight migration reasons. Kabinna et al. [29] identified 49 logging library migrations in 223 Apache Software Foundation projects and found that flexibility and performance are the primary drivers. He et al. [22] analyzed the prevalence, trends, and rationales of the library migration phenomenon occurring in the Java ecosystem on a large-scale dataset (3,163 migration commits from 19,652 projects). They discovered four dominant domains (logging, JSON, testing, and web service) and 14 reasons for Java project migrations. Gu et al. [20] compared self-admitted library migrations (SALMs) occurring in the Java, JavaScript, and Python ecosystems.

Various methods have also been proposed to recommend alternative libraries, leveraging the historical migration data from software repositories. These methods tap into the collective

wisdom of developers' decisions to migrate to alternative packages [5, 46, 47]. However, they face challenges such as low recall or low precision in identifying suitable replacements. Ouni et al. [39] explored a search-based optimization approach that employs the non-dominated sorting genetic algorithm to balance multiple objectives in library recommendation. Taking it a step further, He et al. [24] transformed the recommendation problem into a ranking problem and devised some more subtle metrics to improve the efficiency and accuracy of recommendations. To avoid undesired suggestions, Mujahid et al. [37] proposed an automated approach to identify npm packages in decline and suggest superior alternatives. Recently, learning-based techniques have emerged. Di Rocco et al. [12] proposed a transformer-based approach that recommends both third-party library (TPL) and API migrations to support coupled library–code refactoring. Sang et al. [44] developed an attributed multiplex learning method that integrates multi-level textual and relational data for analogical library recommendation. Large Language Models (LLMs) have also been explored for migration assistance. Almeida et al. [2] first evaluated ChatGPT for API migration tasks under zero-shot, one-shot, and chain-of-thought prompting. Similarly, Islam et al. [27] assessed multiple modern LLMs, demonstrating their capability to migrate Python code with minimal supervision. Han et al. [21] presented the LibRec framework that integrates RAG with LLMs, and provided the LibEval benchmark for evaluation.

Although there has been significant research on library migration in PLs that have central package hosting platforms (e.g., Maven for Java [3, 5, 20, 22–24, 46, 47], npm for JavaScript [7, 20, 37], and PyPI for Python [20, 26, 29]) and considerable datasets on these migrations exist [3, 20, 22], migrations in C/C++ ecosystem have received comparatively less attention. No specific tool or methodology for migration mining and recommendation is currently available in the C/C++ ecosystem, either. Even for LLM, it is hard to collect related information on C/C++ libraries and recommend appropriate targets due to its fragmented ecosystem. In this study, we define the **C/C++ ecosystem** as software projects primarily using C/C++ as the main language and their dependent libraries.

Partially inspired by similar work on other languages, in this study, with a rule-based migration mining algorithm, we construct the first large-scale C/C++ migration dataset across seven package management tools to conduct an empirical study on the prevalence, domain, targets, and rationale of migrations in the C/C++ ecosystem.

3 Approach

To construct a dataset that can reflect the landscape of C/C++ library migrations, we first select C/C++ projects that adopt certain package management tools (Section 3.1). Then, we parse the dependencies of these projects utilizing specific parsers (Section 3.2). After that, we identify SALM rules based on the change of dependencies. Finally, we apply the rules to the entire history of the change to construct the final migration dataset for the C/C++ ecosystem. (Section 3.3) The approaches we used for subsequent data analysis are described in each research question, respectively. Figure 1 presents the logic flow of the approach adopted in this study.

3.1 Project Selection

A dataset covering all the C/C++ projects is ideal for exploring our research questions. To achieve a comprehensive data scope, we begin with *World of Code* (WoC), a large-scale open-source dataset and platform[1]. WoC provides comprehensive and unified access to almost all public Git repositories on dozens of platforms, including GitHub, GitLab, Bitbucket, and so on. It organizes data based on the Git data structure (blob, tree, and commit) and stores mappings of other types of data, such as authors, projects, files, and so on. Given the breadth of PMTs identified by Tang et al.[45], our study

[1]https://bitbucket.org/swsc/overview/src/master/

narrowed its focus to seven PMTs, as shown in Table 1, for three reasons. First, they are widely used in C/C++ projects [45] and cover different types of PMTs. For example, Deb is widely adopted by projects in the operating system domain, which constitutes a key and unique application domain of C/C++, and whose importance and popularity differ from those of other programming languages. Conan, Vcpkg, and Xmake reflect emerging PMTs in the C/C++ ecosystem, aiming to provide package hosting platforms. Second, they adopt structural file formats such as JSON and config as configuration files, which allows us to parse dependencies with high accuracy. Third, other PMTs are either rarely used, such as Buck [45]. Or it is difficult to accurately parse dependencies specified in them, such as CMake and Autoconf, which threaten the validity of the results. CMake or Autoconf are scripts and can not often be parsed accurately by static analysis. Developers can also download libraries directly from the Internet, and it is hard to determine the nature of these libraries.

Specifically, we start from the World of Code database (version V; snapshot March 2023). We consider non-forked C/C++ projects that use any of seven package-management tools (PMTs), yielding 21,753 projects. These projects contain one or more configuration files in which developers declare the names and versions of required libraries (e.g., conanfile.txt, vcpkg.json, .gitmodules). Some unused libraries may still be declared, which constitutes a threat to validity (Section 8.2.2). Requiring at least one active month per project leaves 19,943 projects. Summary statistics for the selected projects are reported in Table 2.

3.2 Dependency Parsing

We design a parser for each dependency configuration file adopted by the seven PMTs. The details of each parser are listed in Table 1. We use different parsing approaches to resolve specific dependency fields for different PMTs. For example, we use regular expressions to parse the `dependency` field of `meson.build` (`dependency('libplacebo', version: '>= 3.110.0', required: false)`), and just match declaration patterns to parse `Requires` field of `*.pc` (`Requires: actionlib_msgs std_msgs trajectory_msgs`).

For validation, ten different configuration files are sampled for each PMT, totaling 70. Two authors with three years of C/C++ development experience independently checked all seventy files and labeled the dependencies introduced by certain PMTs. We use Krippendorff's alpha with MASI distance [30] to measure the inter-rater agreement of dependencies and get a result of 0.96. After discussion, all disagreements are resolved and the label result serves as our ground truth. Our dependency parsing method obtains a precision of 84% and a recall of 90% on the ground truth. Most false positives are caused by multiple choices in if-statements or local projects built by developers. Most false negatives are caused by the declaration of dependencies in script files (`.py/.lua`). However, these false positives will be excluded in our following approach to identifying SALMs by matching migrated libraries in commit messages. For the false negatives, static analysis is hard to deal with all of them. To mitigate this problem, we manually sample and check the configuration files to collect as many regex rules as possible and parse the abstract syntax tree (AST) to reduce the omission of dependencies.

3.3 Library Migration Identification

To obtain a comprehensive dataset containing all the possible C/C++ library migrations as accurately as possible, we go through three phases. First, we extract dependency changes in selected projects to generate candidate migrations. Second, we identify *self-admitted library migrations (SALMs)* [20] from these candidates to achieve migration rules; third, we utilize the migration rules to rematch the candidate migrations we have collected, thereby obtaining a more comprehensive migration dataset.

Table 1. Configuration Files Parsed in Our Method

PMTs	Configuration Files	Parsing Approach	Dependency Field(s)
Meson	meson.build	regex	`dependency`
Xmake	xmake.lua	regex	`add_requires`
Deb	debian/control	regex	`build-depends`, `depends`
Conan	conanfile.txt/py	regex/AST	`build_requires`, `requires`
Vcpkg	vcpkg.json	JSON parser	`dependencies`
Pkg-config	*.pc	pattern match	`Requires`
Git submodule	.gitmodules	pattern match	`submodule`

3.3.1 Candidate Migration Generation. Commits to C/C++ configuration files suggest changes in C/C++ configuration, indicating possible changes on dependencies, i.e., possible migrations. We utilize the WoC dataset, specifically version V, compiled in March 2023 [33], to mine commits C related to selected C/C++ configuration files in projects $\mathcal{P}$, along with their pre- and post-modification blobs.

Through static analysis mentioned in Section 3.2, we parse the blob contents before and after each commit to identify the introduction or removal of third-party C/C++ libraries. For each commit $c \in C$, developers may introduce new libraries (L_c^+), remove old libraries (L_c^-), or both. All these involved libraries constitute the set of libraries $\mathcal{L} = \bigcup_{c \in C} L_c^+$ (Note that $\bigcup_{c \in C} L_c^- \subseteq \bigcup_{c \in C} L_c^+$, because a library must be introduced first before it is removed). We consider only commits C_{cand} with both library introduction and removal (**candidate commits**), i.e., $C_{cand} = \{c \mid c \in C \wedge L_c^- \neq \phi \wedge L_c^+ \neq \phi\}$. Then, we generate pairs of libraries that are removed and introduced for each c as **candidate migration commits** (denote as $\mathcal{M}_{cand} = \{\langle c, l_1, l_2 \rangle \mid c \in C_{cand} \wedge l_1 \in L_c^- \wedge l_2 \in L_c^+\}$), suggesting potential library migrations. 4,922,903 candidate migrations are generated in this step, corresponding to 45,133 candidate migration commits.

3.3.2 SALM-based Migration Rule Identification and Filter. A candidate migration m_{cand} is considered as a migration m only when it is confirmed by **migration rules**. We followed previous work[20] to identify migration rules based on self-admitted library migrations (SALMs). In SALMs, developers explicitly document migration behavior in commit records so that accurate migration rules can be obtained automatically by matching library names. For PMT migrations, we replace the match of library names with the match of PMT names. However, the existing migration mining algorithms suffer from certain limitations in our study.

First, the naming conventions of different PMTs pose challenges in matching C/C++ library names in commit messages. Each PMT has its own rule and name strategy. For different PMTs, we adopt corresponding strategies to ensure the accuracy of migration identification (e.g., drop `lib-`, `-dev`, or extract library name from git URL).

Second, similar package names can lead to numerous incorrect migration pairs. Gu et al. partially mitigated this issue by aggregating similar packages into super libraries. However, this approach needs a lot of manual effort and results in the loss of detailed information about individual packages. In our work, we significantly reduced these false positives by matching the most similar package names, thereby further improving the accuracy and automation of migration identification.

We further eliminated library update cases from the migration results. For example, C/C++ libraries in Deb often have the version number appended to the library name during release

Table 2. An overview of our dataset

	Projects				Dep changes	Migrations		
PMTs	Total Num	Active Months*	Commits*	Files*	Commits	Commits	Instances	Rules
Conan	552	27	164	107	2,200	224	266	104
Vcpkg	241	40	343	175	2,703	92	96	53
Meson	500	90	1348	291	1,960	253	279	105
Xmake	121	26	231	158	364	5	6	5
Pkg-config	29	31	407	583	81	0	0	0
Gitsubmodule	11,067	48	415	161	18,927	113	123	86
Deb	7,433	113	379	263	18,968	1,050	1,421	364
Total	19,943	64	408	197	45,133	1,737	2,191	717

* The statistical data given in the table is the median.

(e.g., `libmutter-2`, `libmutter-3`). Candidate migration involving only version number changes should be identified as library updates rather than migrations. However, we reserve major version updates similar to those from Qt4 to Qt5, which is a complicated transition from one collection of libraries to another (e.g., `libqt4-dev` to `qtbase5-dev`, `qttools5-dev`, and `qttools5-dev-tools`). In Gitsubmodule, developers may also change another git URL with different owner names yet the same repository name (i.e., a fork project). We detect these candidate migrations between fork or direct version number changes by rules and exclude them as trivial migrations. The candidate migration from `debhelper` to `debhelper-compat` in Deb is also excluded because this change is a constraint on versions of `debhelper` instead of migration between build-related libraries. 717 migration rules $\mathcal{R}$ are identified by SALMs in this step.

Due to the lack of publicly available datasets for C/C++ migration, the validation of our mining algorithm is conducted through a manual review process. 251 rules out of 717 are randomly sampled (0.95 confidence level and 0.05 margin of error), and two authors first check whether the commit messages document a migration, and determine whether there is functional similarity between two libraries through experience or online search. If there is uncertainty, they will read the modified code to determine if there is an API mapping between the two libraries. The final labels result in a Krippendorff's alpha of 0.87. A 96% precision is obtained after resolving all disagreements.

3.3.3 Migration Dataset Construction. Migration rules $\mathcal{R}$ identified from SALMs are used to confirm whether candidate migrations $\mathcal{M}_{cand}$ correspond to **migration instances** $\mathcal{M}$, i.e., $\mathcal{M} = \{\langle c, l_1, l_2\rangle \mid \exists \langle l_1, l_2\rangle \in \mathcal{R}, \langle c, l_1, l_2\rangle \in \mathcal{M}_{cand}\}$. Similarly, we denote **Migration Commits** as $C_m = \{c \mid \exists \langle c, l_1, l_2\rangle \in \mathcal{M}, c \in C_{cand}\}$. This approach is based on the inference that a commit has also conducted a migration if a migration rule occurs in its dependency changes, even if developers may not document it in its records. Through applying migration rules to candidate migrations and dropping the duplicates, 2,191 migrations ($\mathcal{M}$, corresponding to 1,737 migration commits) are obtained and serve as the dataset for our subsequent analysis. The statistics of projects and distribution of migrations among seven PMTs are shown in Table 2.

4 RQ1: How common do C/C++ migrations occur?

The complex nature of dependency management of C++ brings uncertainty to the nature of library migration. In this question, we want to know the prevalence of migrations in C/C++, compare its differences with other languages, and get deeper into the differences among different PMTs.

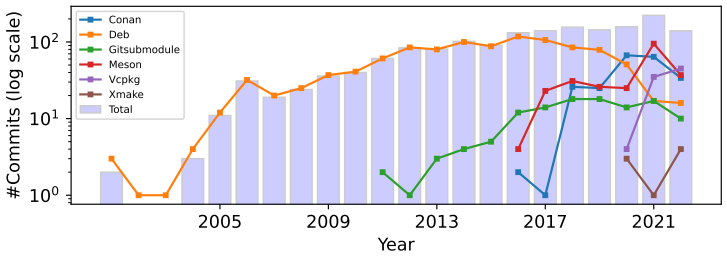

Fig. 2. Number of migration commits in C/C++

4.1 RQ1.1 How common do C/C++ migrations occur in different package management tools?

4.1.1 Method. Based on our dataset, we calculate both the number of migrations and the number of projects with at least one migration to describe the lower bound of the number of migrations. For projects, the set of projects that have ever conducted migrations is denoted as $\mathcal{P}_m = \{p \mid p \in \mathcal{P} \wedge \exists c \in C_p, c \in \mathcal{C}_m\}$ if we let C_p be the set of commits for p. We compare the number of $\mathcal{C}_m$, $\mathcal{M}$, and $\mathcal{P}_m$ among the seven PMTs. Finally, we plot the trend of overall migrations $\mathcal{M}$ and migrations in each PMT by commit timestamps.

4.1.2 Results. As shown in Table 2, 6.68% of projects (1,333 / 19,943) have conducted at least one migration within the studied scope, and 3.85% of candidate commits (1,737 / 45,133) are migration commits. The number of migrations differs vastly across PMTs. Deb contributes 65.11% of migrations (868 / 1,333), which is far more than those in other PMTs because of its long history and wide usage. Conversely, only 123 migrations are identified in Git submodule, despite its history spanning over a decade, where 31.32% of candidate commits (5,927 / 18,927) are just changing another fork repository and thus excluded from this study. Xmake presents the fewest migrations (only six), and we identified no migrations in Pkg-config. That may be because only a few projects adopting these two PMTs are analyzed, or developers did not explicitly document the migration in commit messages.

To better understand the migration trend in C/C++ PMTs, we plot the longitudinal trend of migration commits (histogram in Fig. 2) and projects with ≥ 1 migrations (not shown because the trend is highly similar to the number of migration commits) in log scale, which starts in 2001 and ends in 2023, because the data trim in March 2023. We find that both the total number of migration commits and projects increased slowly (until 2016) and finally remained relatively stable. The lines in Fig. 2 with different colors demonstrate the trend of migration commits across different PMTs. We find that although the overall trend has remained relatively stable since 2016, the amount of migrations fluctuates significantly across different PMTs. It is worth noting that migration commits in Meson sharply increased in 2020 (when *Meson 0.54.0* was released, adding support for the CMake project and library), quickly surpassing the rapidly declining Deb, making Meson the PMT with the most migrations.

Comparison with other PLs:

As a mature programming language, the trend of migrations in C/C++ PMTs is similar to Java [20]. They both experienced a gradual increase and then a stabilizing trend. On the one hand, this stabilized migration trend reflects the maturity of both C/C++ and Java ecosystems, where major libraries are well-established. In newer languages like Python and JavaScript, libraries are often subjected to significant overhauls, introducing new features and deprecating old ones at a faster pace [20]. The stable feature of C/C++ libraries facilitates maintenance and long-term planning, which also enhances the feasibility of automated migration tools.

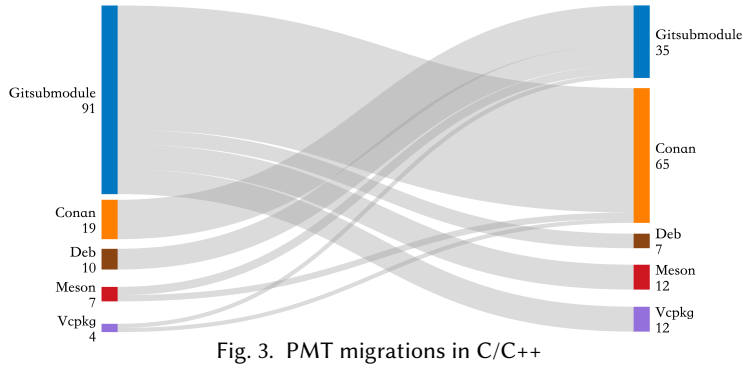

Fig. 3. PMT migrations in C/C++

On the other hand, despite the long development history of C/C++, its dependency management has been continuously evolving mostly since it does not have a universal package hosting platform (in other words, it has fragmented package management tools). To tackle the problem, new tools have emerged, e.g., Meson, Conan, and Vcpkg. Migrations in these PMTs keep growing because libraries under them are evolving rapidly, leading to a more dynamic migration landscape, which behaves similarly to Python. These specific PMTs are worth paying attention to and may require support for migration automation-related tools.

Summary for RQ1.1:

Among the 19,943 projects, 1,333 (6.68%) have conducted at least one migration. The overall amount of migration commits in C/C++ increased slowly and remains stable with a slightly decreasing trend, the same as Java. However, the number of migrations and projects that conducted migrations in emerging package management tools (Meson, Vcpkg) is increasing, while the migration in those established (Deb), although initially higher, is experiencing a decline.

4.2 RQ1.2: How common do C/C++ migrations occur across different package management tools?

4.2.1 Method. Due to the lack of a universal way for C/C++ dependency management, developers may also **migrate their PMTs** during the development of projects. In this part, we calculate this migration with a similar identification approach as SALMs, replacing 'libraries' with 'PMTs'. And we define such migrations as **PMT Migrations** (denoted as $\mathcal{M}_{PMT} = \{\langle c, PMT_1, PMT_2\rangle \mid c \in C_{cand} \wedge l \in L^{-}_{c,\ PMT_1} \wedge l \in L^{+}_{c,\ PMT_2}\}$). 306 PMT migration commits (291 confirmed manually by checking the modification of corresponding dependency configuration files) are identified, and we drop the duplicated PMT migrations that occur in a single project. The duplicate results occur because a project may migrate multiple libraries in a migration commit. And we only care about changes in PMT here, rather than specific libraries. 141 projects with PMT migration commits and 15 PMT migration rules are identified. 15 projects mentioned why they changed their PMTs. This type of migration is significantly different from the conventional migrations typically discussed; therefore, here we only discuss its frequency of occurrence and underlying causes.

4.2.2 Results. Fig. 3 shows the PMT migration rules that occur more than once. The most frequent migration rule is from Git submodule to Conan (60 projects). 47.5% (67 / 141) of projects switch submodules to different package managers to mitigate the effort of dependency management. Opposite migrations also exist, such as 19 projects changing Conan to Git submodule. The reason for such a phenomenon may be two major characteristics of the C/C++ ecosystem: one is that

each PMT employs a unique management strategy, resulting in varying levels of compatibility and usability; another is the widespread practice of code cloning in C/C++. Using Git submodules, developers can maintain their git branches or keep synced with the latest snapshots.

Summary for RQ1.2:

141 C/C++ projects are observed to have changed their package management tools (compared to 1,192 projects that do not change). While migrations from Git submodules to Conan are most frequent, the opposite migration path is also observed.

5 RQ2: In which domains do C/C++ migrations occur?

Migration domains vary across different ecosystems, reflecting the application of PL, as well as its evolution in ecology [20]. This question aims to understand the most common domain for C/C++ in which migrations occur and reveal frequent migration domains that require more attention.

5.1 Method

Our analysis covers all 416 libraries related to migrations that we have identified. Given the decentralized sources of C/C++ libraries, developers may download them from platforms such as Conan, vcpkg, or directly from GitHub. We retrieve descriptions or README.md of these libraries from respective sources to classify them into different categories. Libraries from Conan/Vcpkg are obtained from certain hosting platforms (conan.io, github.com/vcpkg). Libraries from meson/xmake/gitsubmodule are obtained from GitHub or download URLs. Libraries from Deb are obtained from the Debian package hosting platform. 362 of 416 libraries are obtained from certain package hosting platform, and the remaining libraries are obtained through manual search on the Internet to obtain their repository URL. Then, we randomly sample 100 libraries from the 416 libraries for convenience. One author labels them with different domains and generates a coding book. Another author uses this coding book to again label the 100 randomly selected libraries, resulting in a Krippendorff's Alpha of 0.84. Second, after resolving the disagreements and ensuring clear and accurate expression in the coding book, two authors label the remaining libraries independently. Finally, we get 19 distinct domains in total. These domains cover common results as other languages (e.g., network, database, testing) and C/C++ specific domains (e.g., OS development). Fig. 4 presents the domain distribution of migrations. Domains with fewer than 20 migrations are merged and displayed as 'Other' in the figure. We also visualize three sub-graphs for the top three domains using Sankey diagrams [43] to show how migrations flow in these domains.

5.2 Results

Two domains (Build and GUI) occupy nearly half of the migrations (48.08%), followed by OS development, which is also an essential application domain of C/C++, accounting for 7.92%. However, Testing and Logging, which are critical migration areas in other ecosystems, only account for 4.56% and 1.06%, respectively. The details of the top three domains with the most migrations are as follows:

GUI: GUI occupies the highest proportion (29.06%) in C/C++ migrations. In this domain, migrations may be primarily driven by technological innovations. The most frequent migration within this domain is from Qt4 to Qt5 (Fig. 5). The "upgrade" from Qt4 to Qt5 is accompanied by significant performance optimization and usage updates. It led to the fragmentation of the original package (*libqt4-dev*) into multiple components (*qtbase5-dev*, *qttools5-dev*, *qttools5-dev-tools*), which is why we regard this as a kind of migration. Another typical migration is from *libhandy-1* to *libadwaita-1*. The two libraries aimed at providing adaptive, responsive UI widgets for GNOME applications,

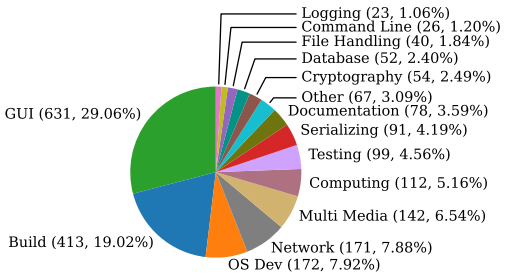

Fig. 4. Distribution of domains for migrations in C/C++

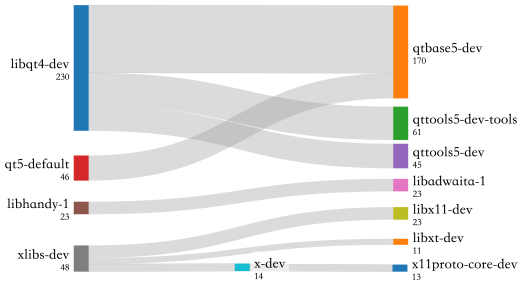

Fig. 5. Migration subgraph for GUI libraries

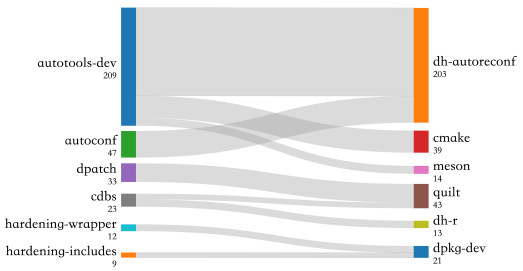

Fig. 6. Migration subgraph for Build libraries

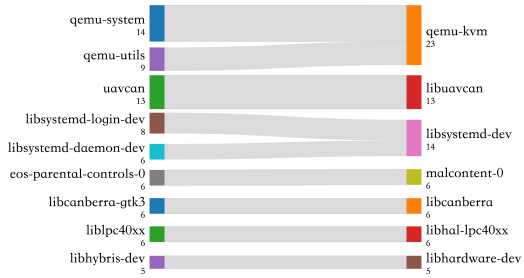

Fig. 7. Migration subgraph for OS Development libraries

while libhandy-1 is rooted in the GTK3 era, and libadwaita-1 represents the modern, GTK4-based direction of development.

Build: Build libraries refer to the libraries or tools used during packaging or distributing. Their dominance of the migration domain in the C/C++ ecosystem may be due to the complexity of the build process of Debian packages. The most frequent migrations are ⟨*autotools-dev, dh-autoreconf*⟩ (156 migrations). *dh-autoreconf* was introduced to automate the regeneration of *Autotools* configuration files during the Debian package build process. The Debian packaging system is a mature and widely adopted method for managing operating system libraries, and it also handles numerous C/C++ libraries. By eliminating the need for static helper files provided by *autotools-dev*, it simplifies maintenance and enhances cross-platform compatibility, which led to its gradual migration. Other migrations in this domain (⟨*autoreconf, dh-autoreconf*⟩, ⟨*dpatch, quilt*⟩) also show the essence of functionality and ease of maintenance for the build process.

OS Development: In this domain, developers migrate libraries on process/memory/session management, file system, or drivers. Migrations in this domain may be the result of specific functional needs for platforms and architectures. Frequent migrations are ⟨*libsystemd-login-dev, libsystemd-dev*⟩ and ⟨*qemu-system, qemu-kvm*⟩. The first migration resembles a consolidation of functionalities, while the second is tailored for another runtime environment. There are also migrations involving libraries across different interaction layers with hardware (⟨*liblpc40xx, libhal-lpc40xx*⟩), where *liblpc40xx* provides direct hardware-level APIs, while *libhal-lpc40xx* leverages those low-level functions to offer a more accessible and unified interface for application development.

Comparison with other PLs: The domains in which C/C++ library migrations occur differ significantly from those of other PLs. Unlike Java, JavaScript, and Python, where Testing, Network, and Serialization are the most frequently migrated areas [20, 22], the predominant domains for

migrations in C/C++ are GUI and Build. The differences in migration domains can be attributed to their domain-specific applications. This shift also highlights the increasing significance and technical evolution in packaging and high-performance graphics rendering for C/C++ projects. We can also see that the selection of migration alternatives for C/C++ developers tends to be consistent. One source library usually corresponds to no more than three target libraries (Fig. 5). While for Java developers, one third of source libraries correspond to two or more targets [22]. The details will be discussed in Sec. 6.

Summary for RQ2:

The domains with the most frequent library migrations in the C/C++ ecosystem are GUI and Build, underscoring both the complexity of the build process and the critical role C/C++ plays in graphics development. Compared to other PLs (Java, JS, Python), C/C++ developers tend to migrate Build and application-oriented libraries (GUI) rather than basic libraries like testing, logging, and serializing.

6 RQ3: How do developers choose their migration targets?

In RQ2, we identify different distributions and directions of migration targets in different domains. This question attempts to go further to characterize the diversity of migration targets in the C/C++ ecosystem.

6.1 Method

Library migration can be represented as a weighted directed graph where nodes represent libraries, edges represent a migration relationship between the two software packages. Weights represent the number of commits undergoing the migration rules represented by edges. Previous studies analyzed the differences in weighted degree (i.e., differences between weighted in-degree and out-degree) by $Flow(l)$ to demonstrate the extent to which a library is more likely to be adopted or abandoned [22]. However, the distribution of the weighted degrees (i.e., diversity of target libraries) is neglected, which is another important aspect of selecting targets. Inspired by the approach adopted by prior studies [20, 22, 26], we extend the features of the migration graph to calculate both the distribution and difference of weighted degree, referred to as **diversity** and **directionality**, respectively, to analyze the distribution of migration targets in C/C++. To compare with other PL ecosystems, we calculated the results using publicly available migration datasets in Java, Javascript, and Python [20, 22].

Diversity Choosing an appropriate target is crucial for library migration, especially from multiple candidates. The more candidates there are, the more challenging the selection may be. We use $Entropy(l)$ to describe the diversity of candidates for a source library l (Equation 1). Entropy is used to quantify the uncertainty in information. Here, a high entropy of a source library indicates that developers exhibit divergent perspectives on the alternatives for this library, and more importantly, the distribution of their choices tends to be rather uniform. If the entropy of a source library is zero, all developers choose the same target.

$$Entropy(l) = -\sum_{l_t \in \mathcal{L}} p(l, l_t) \log_2 p(l, l_t) \tag{1}$$

$$p(l_1, l_2) = \frac{|\{\langle c, l_1, l_2\rangle \mid \exists c \in C_m, \langle c, l_1, l_2\rangle \in \mathcal{M}\}|}{|\{\langle c, l_1, l\rangle \mid \exists c \in C_m \wedge l \in \mathcal{L}, \langle c, l_1, l_2\rangle \in \mathcal{M}\}|}$$

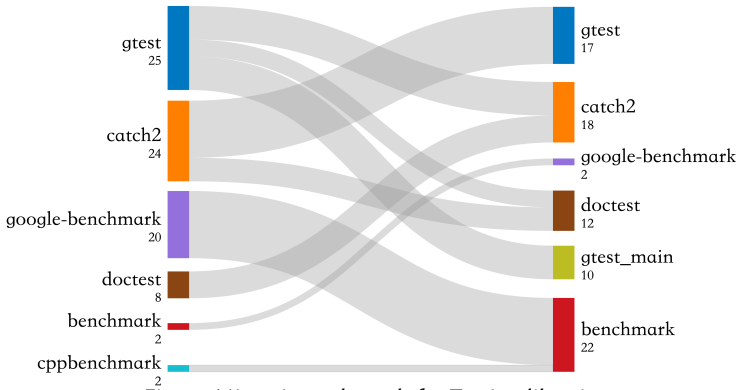

Fig. 8. Migration subgraph for Testing libraries

Directionality Some libraries are frequently deprecated or adopted (e.g., *jsonformoderncpp* and *nlohmann_json*). However, some libraries might be adopted by a subset of developers while being abandoned by others (e.g., *gtest* and *Catch2*). Understanding the adoption-to-abandonment rate of library migrations, i.e., the directionality of migrations, may inform us about which libraries have been phased out and which ones remain in a competitive state within the ecosystem.

Following previous studies [22, 46], we use $Flow(l)$ to describe the directionality of a source library l (Equation 2). Here, we take the absolute value of the $Flow(l)$, focusing only on the magnitude of directionality rather than distinguishing whether libraries are adopted or abandoned. If $Flow(l) = 1$, it indicates the library l is always adopted or abandoned.

$$Flow(l) = \left| \frac{deg^-(l) - deg^+(l)}{deg^-(l) + deg^+(l)} \right| \tag{2}$$

6.2 Results

6.2.1 Diversity of target libraries. To illustrate entropy clearly, we visualize a sub-graph for a typical migration domain (Fig. 8). It shows a completely different migration situation from Fig. 7, where a source library corresponds to multiple target libraries and a library is abandoned in one migration while adopted in another. In this subgraph, $Entropy(gtest) = 1.52$, $Entropy(catch2) = 0.60$, and $Entropy(doctest) = 0$. It indicates that the migration target of *gtest* exhibits a higher degree of uncertainty, as it splits into three targets (*catch*, *doctext*, and *gtest_main*), with *doctest* and *gtest_main* having similar proportions (both respond to 10 migrations). Developers may not form a consensus of the alternatives of *gtest*. While for *catch2*, it migrates to two targets, *gtest* (17) being superior to *doctest* (7). And *doctest* has only one migration target *catch2*.

Furthermore, we calculate the entropy of all source libraries in C/C++. Fig. 9 shows its extremely distorted distribution of $Entropy(l)$. For 83.46% of the source libraries (449 / 538), $Entropy(l) = 0$, which means that developers only have one alternative option (i.e., one-to-one migration). For the remaining libraries with multiple targets, 17% of them (16 / 89) have an entropy lower than 0.88 (calculated by $|\mathbf{L_t}| = 2$ and $p(l, l_1) = 0.7$). It means that although there may be more than one alternative, at least 70% migrations still correspond to the same target library. Developers may have reached consensus or followed existing practices when conducting such migrations.

However, 44 libraries (8.18%) have relatively high entropy (0.97 for $|\mathbf{L_t}| = 2$ and 1.57 for $|\mathbf{L_t}| = 3$), indicating a huge difference in target library selection. Among these libraries, GUI and Testing libraries need attention. For example, for the Graphics library *glew*, 11 migrations choose *glad* to replace it. However, 8 choose *epoxy* and others choose *sld2*, *glbinding*, and so on. (Testing libraries

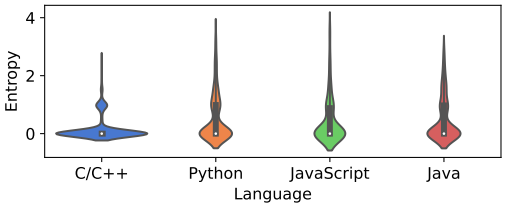

Fig. 9. Distribution of entropy for migrations in four ecosystems

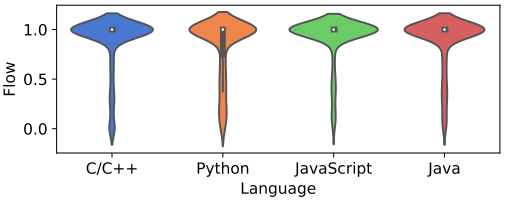

Fig. 10. Distribution of flow for migrations in four ecosystems

are shown in Fig. 8) In this case, developers need to consider their requirements carefully and choose an appropriate alternative.

6.2.2 Directionality of the library migrations. We calculate the *Flow* of each library with Eq. 2. The value range of *Flow* is 0 to 1. The larger the flow, the more commonly the library is adopted or abandoned. It means that there may be alternatives superior to this library, so it is always abandoned. Or this library itself is superior to other libraries, so it is always adopted. Fig. 10 shows the distribution of $Flow(l)$ for source libraries l. Similar to $Entropy(l)$, the highly skewed distribution of migration flows indicates that the vast majority of libraries are always adopted or abandoned. 79.60% of migration rules are entirely unidirectional, suggesting that the target libraries are superior to the ones they supplant in all aspects.

Comparison with other PLs: For the diversity that has not been studied before, we calculated the results using publicly available datasets [20, 22]. Fig. 9 clearly shows that C/C++ exhibits a distinct tendency towards one-to-one migrations (83.46%), in significant contrast with migrations observed in Python (63.32%), JavaScript (69.57%), and Java (65.28%). This huge difference does not change obviously, even if we exclude libraries with migrations that occur fewer than five times. Factors inherent to the C/C++ ecosystem may contribute to this phenomenon. First, the C/C++ ecosystem has been characterized by well-established libraries such as *boost* and *qt*. These libraries are highly optimized and widely used, where developers may have formed a consensus. The same source may release different libraries for various requirements or architectures. Secondly, the complexity in managing dependencies makes C/C++ developers conservative when choosing dependencies. They always follow established standards and best practices in communities. Thirdly, the fragmentation of package management platforms may impose certain limitations on C/C++ developers when selecting software packages. For the distribution of $Flow(l)$, JavaScript migrations (84.68%) exhibit the highest degree of unidirectionality, while Python migrations (71.43%) demonstrate the lowest. This may be the result of the characteristics of ecosystems. Libraries in the Python ecosystem face intense competition, whereas the JavaScript ecosystem exhibits a high rate of library deprecation [20].

Summary for RQ3:

Although some libraries have multiple options in domains like GUI and Testing, for 83.46% of C/C++ libraries, migrations are one-to-one. 79.60% of migration rules are entirely unidirectional in C/C++. The much higher one-to-one and unidirectionality proportion compared to other languages suggests that C/C++ developers may have formed a consensus on migration.

Table 3. Definitions and distribution of migration rationales in C/C++. The rationales marked with † have extended definitions compared to previous study [20, 22], and the extended definitions are highlighted with underlines and **bolded**.

Rationale	Definition	Frequency
Source Library	Problems in the source library	11 (16.18%)
Deprecation	The source library is inactive, not maintained, will be deprecated in the future, or not recommended to use officially.	3 (4.41%)
Bug or issue†	1) Source library has bugs or emits warnings. 2) The source library is **not stable** or **can not be compiled successfully** on certain platform.	8 (11.8%)
Target Library	Advantages in the target library	30 (44.12%)
Functionality	1) The target library has new and powerful functionalities for the project to implement its desired features. 2) The target library has more functionalities and makes the project more adaptable to future changes.	14 (20.59%)
Usability	1) The target library is easy to install, use, or maintain; the target library has better documentation. 2) Using the target library can bring ease of implementation and cleaner code.	4 (5.88%)
Performance†	1) Using the target library can improve runtime performance. 2) Using the target library results in **less compiling effort**.	5 (7.35%)
Activity	Although the source library is under maintenance, the project still chooses to use a more recent, well-maintained, or stable dependency.	5 (7.35%)
Popularity	The target library is more widely used or complies with industrial standards/ecosystem best practices.	1 (1.47%)
Size/Complexity	The target library is simpler or results in a smaller binary size.	1 (1.47%)
Project Specific	Project specific requirements	27 (39.71%)
Integration	1) The project needs to integrate with different operating systems, platforms, or architectures. 2) The project has to resolve conflicts between dependencies.	15 (22.06%)
Simplification†	1) The project migrates to reduce the number of dependencies. 2) The project **no longer needs certain functionality** and replace relevant libraries. 3) The project migrates to **unify the package hosting platform**.	12 (17.65%)

7 RQ4: Why do developers conduct C/C++ migrations?

7.1 Method

Understanding why developers migrate can help them avoid potential migration issues and enable related tools to recommend appropriate alternatives. SALMs identify migrations based on the clarification in commit messages. However, not all developers choose to explain why migrations are conducted. Given the large dataset in our work (2,191 migrations), it is challenging to check and classify each message manually. Therefore, we first filter the commit messages by some keywords that are often used to describe migration reasons (e.g. *because, for, so that, deprecate, better, can, help, need, since, due to, result in, avoid*). Such keywords are summarized and refined from previous work [20, 22]. After filtering, we get 316 records containing those migration-related keywords. One author first labels these records based on previous rationale framework [20, 22] and generates a coding book for C/C++ migration rationales for the first iteration. Then, two authors independently label migration-related messages using this coding book. Since a message can involve multiple reasons, we use Krippendorff's alpha with MASI distance to measure the inter-rater agreement [30]. The Krippendorff's alpha of the second iteration is 0.86, which is higher than the recommended score of 0.8 [30]. We finally got three themes and ten sub-themes.

7.2 Results

Table 3 shows the migration rationale in C/C++, including 10 categories among three themes (*source library*, *target library*, *project-specific*), summarized from 68 migration commits with documented reasons. The second column describes the definition of each category and theme. The third column calculates the number of projects that migrate due to certain reasons. Four new types are discovered in C/C++ compared to the framework proposed in previous studies [20, 22], as highlighted with † in

Table 3. To avoid repetition, we only show explanations for rationales already covered in previous studies in the definition column of Table 3. The newly extended rationales are as follows:

Bug or issue under *Source Library*: The source library is not stable or can not be compiled successfully on certain platforms. C/C++ libraries are usually used in operating systems, where a variety of distributions necessitate library adjustments and compatibility measures. Therefore, libraries may not be stable or exhibit bugs on certain platforms. For example, *`gcr-4` is not yet stable for gnome 43* [18].

Performance under *Target Library*: Using the target library results in less compiling effort. Compiling effort and time is a factor influencing developers' choices among libraries due to the lengthy compiling time of C/C++. For example, *Removed unused `OpenSSL` dependency (replaced with `CryptoPP` due to static linking and compile times)* [16].

Simplification under *Project Specific*: The project migrates to unify the package hosting platform. Given the variety of C/C++ PMTs, some developers try to change and unify their dependency management methods. For example, they might choose to only use libraries from Conan or Vcpkg, e.g., *Use cmark for commonmark parsing (because `md4c` isn't in Vcpkg)* [19]. This reason highlights the complexity of C/C++ dependency management. Developers try to alleviate this problem by adopting libraries from the same package hosting platforms. Another reason for simplification is that the project no longer needs certain functionality and replaces relevant libraries, e.g., *Use plain `libcanberra` instead of `libcanberra-gtk3`. We no longer use the gtk-aware ca_context, the dependency can be lowered now* [15].

The third column of Table 3 presents the distribution of rationales. The complex development environment and cross-platform requirements of C/C++ projects result in a high proportion of migration due to project-specific reasons (39.71%). Advantages in the target library (44.12%) are another main reason, driven by the need for new features in certain domains (e.g., GUI) and performance requirements of C/C++.

Comparison with other PLs: We conduct an initial exploratory comparative analysis using this dataset. When conducting C/C++ library migrations, developers tend to consider more advantages in the target library and project-specific requirements. This preference is aligned with Python [20], but the reasons are distinct. While Python developers need to consider compatibility with both Python 2 and Python 3, C/C++ developers need to consider different OS distributions and architectures. Python is designed for easier use and development, so functionality and usability are favored by developers. In contrast, C/C++ development is often perceived as challenging, with a lengthy compiling time. Migrations in C/C++ usually have clear targets. Therefore, more functionalities, better usability, and less compiling effort may become goals pursued by C/C++ developers. For example, simplify the size of dependency (e.g., ⟨*boost*, *boost-range*⟩). Meanwhile, the proportion of simplification (17.65%) in migration reasons is similar to Java (18.54%) [20]. This similarity can be attributed to the frequent use of these two languages in large-scale projects, where maintaining only essential features and compatible libraries is crucial. An intriguing observation is the C/C++ developers' effort to unify their package management methods through migrations. This indicates that various PMTs indeed pose significant challenges for developers, while consistent package management (e.g., Maven) can greatly simplify dependency tracking.

Summary for RQ4:

C/C++ developers tend to migrate to integrate across various platforms (22.06%), leverage new features of the target library (20.59%), or simplify dependency management (17.65%). Four reasons not discovered by the prior studies are unveiled: stability, less compile time, abandonment of unnecessary functionality, and unification of dependency management.

8 Discussion

8.1 Takeaways

8.1.1 Evolution of package management tools. As discussed before, C/C++ dependency management is notoriously fragmented and complex [36] compared to other languages. To manage the complexity, C/C++ evolves. The number of migrations in emerging PMTs like Meson and Vcpkg is on the rise (RQ1), diverging from the traditional tools that have been staples for many years. While Deb's historical significance and widespread adoption have resulted in its highest number of migrations, the landscape of package management in C++ is evolving with the emergence of modern tools such as Meson and Vcpkg. Most migrations occurring in Deb are building and packaging-related tools. Modern package managers such as Vcpkg and Conan simplify the process of managing dependencies. They offer declarative and intuitive configuration files that streamline the addition, update, and removal of libraries. They are also able to support a wide range of platforms and compilers, and integrate smoothly with modern build systems (CMake) and continuous integration (CI) pipelines. Tools like Meson offer advanced features, improved performance, and better usability compared to traditional methods [35]. However, the migrations between different management tools also haunt developers. We observe that a considerable number of projects adopt more than one tool to manage C/C++ dependencies, which leads to a much higher maintenance cost. Considering the huge effort of migrating PMTs from one to another, we suggest that developers choose a more modern tool to manage their dependencies.

8.1.2 Tools for C/C++ migrations. Researchers have dedicated significant efforts to automated tools for migration recommendation and code adaptation [4–6, 8, 11, 52]. Despite the fragmentation and complexity inherent in C/C++ package management, the highly unidirectional and one-to-one characteristics of library migrations in C/C++ help alleviate some of the burdens when developers select alternatives. Unlike other languages, C/C++ developers can significantly benefit from established best migration practices, reducing the complexity involved in selecting target libraries. One-to-one migration rules allow developers to focus on directly replacing old libraries with new ones that offer better performance, features, or compatibility. To leverage these advantages, the focus of migration or recommendation tools for C/C++ could prioritize mining migration rules and automating code adaptation rather than evaluating which new library to adopt. Package hosting platforms could document best practices, serving as guidelines for similar migration needs. We also observe that some major updates (`Qt`) even refactor the whole library, leading to one library being separated into different new components. Migration from such an update is not easy, and the qt-wiki has even provided corresponding documents [34]. Therefore, such tools could support migrations between two collections of libraries. Developers in certain domains can also adopt modular design principles or leverage automated tools to mitigate the complexity of these migrations.

8.1.3 Adoption of libraries. The migration of libraries within C/C++ projects is often driven by three rationales (functionality, integration, and simplification) from two themes (target library and project-specific reasons) Cutting-edge features, intuitive APIs, extensive documentation, and faster compile times occupy a significant portion. While project-specific concerns vary, dependency conflict or license compatibility should be taken seriously for each project. Moreover, using libraries available on the same package management platform may represent a novel and beneficial approach to dependency management. The rationale for C/C++ migration indicates that developers could consider the specifics of projects (e.g., supported platform, performance requirements, and even PMTs) when adopting a library.

8.1.4 Use of migration dataset. In this work, we adopt an automatic approach to identify migrations in seven C/C++ PMTs and construct a dataset with 2,191 migration records. In the dataset, we

record not only the source library and target library of a migration, but the project that conducted this migration, its URL, and the migration reason as well. Our dataset can be used to recommend target library directly or further investigation for migrations in C/C++. Developers and researchers could access relevant information, such as the modified code snippets, license, and migration reasons, easily via the URL and dataset. They can search for migration reasons for personal demands or directly search the source library to find the appropriate target library because of the one-to-one characteristics of C/C++ migrations. A simple workflow is as follows. For migration recommendation, developers can directly search the source library to obtain its migration targets as the potential target library. Then, they can refer to the corresponding migration commit url and commit message for detailed information. For migration analysis, researchers can directly perform statistical analysis on data, or retrieve certain code modification through the commit url for further analysis.

8.2 Threats to Validity

This section presents our threats-to-validity analysis and the corresponding mitigations, organized by conclusion, construct, and external validity. We omit further discussion of internal validity because it concerns how sure we can be that the treatment actually caused the outcome [38], while we do not attempt causal inference in this study. For each dimension, we (i) identify potential biases rooted in our data and methods (e.g., dependency parsing, sampling, commit-message-based rule extraction, and manual coding), (ii) examine their implications for our findings, and (iii) detail the mitigation strategies we applied.

8.2.1 Conclusion Validity. It concerns how sure we can be that the treatment we used in an experiment really is related to the actual outcome we observed [38]. Our findings rely on descriptive statistics and manual coding, both of which can introduce bias. We mitigate this risk by having two authors independently code the data and reconcile disagreements. Our prevalence estimates (counts of migrations and projects) may underestimate true frequencies, but they provide a conservative lower bound. We verify the accuracy of our approach by randomly sampling. The sample may introduce bias because of statistical bias. To mitigate this threat, we conduct sampling according to 95% confidence level and 5% margin of error to ensure the adequacy of the sample and the accuracy of the result. We filter the migration reasons by certain keywords, which means we may miss some rationales mentioned by developers with other keywords. To mitigate this threat, two authors independently review prior datasets of migration rationales [22, 29, 47] and extract keyword sets of 20 and 22 terms, respectively. Among these keywords, 18 are identical or highly similar. Each set achieves 100% recall on the prior migration reasons. After reconciling differences and merging similar phrases, we obtain a final set of 24 keywords (please refer to our replication package). Our goal is to maximize recall—that is, to match as many migration reasons as possible—so we intentionally retain a broader keyword set rather than a minimal set of labels for fine-grained rationale categories. The results show that only a small portion of commits are identified as containing rationales by this method. However, nearly half of the commit messages do not always mention details and could be improved [48].

8.2.2 Construct Validity. It concerns the relation between the theory behind the experiment and the observation [38]. Our study relies on parsing dependency profiles to identify C/C++ dependencies [45]. However, an inherent threat lies in the fact that the listed dependencies might not be utilized. This discrepancy arises because dependency files typically enumerate all potential dependencies required for code compilation, but do not necessarily indicate which dependencies are active or relevant during runtime. However, both runtime and development-time dependencies are essential: the former underpin end users' ability to execute the software reliably, while the

latter enable developers to build, test, and maintain it reproducibly; so we have retained all relevant migrations. Migrations from third-party libraries to standard libraries are also excluded because no new libraries are introduced. We utilize static analysis to identify dependencies in C/C++-related configuration files. For some dynamically loaded dependencies, we may miss them. To mitigate this threat, we analyze a large number of projects together with the full set of libraries they used. The observed migration prevalence (6.68%) is comparable to that previously reported (8.98%), indicating that potential omissions have limited impact on the migration rules. In our results, we analyzed the prevalence, domain, target, and reason of library migration in the C/C++ ecosystem. To illustrate the prevalence, we calculate the number of migrations and the number of projects with migrations. This may not reflect the real frequency of migrations. However, the numbers can demonstrate the prevalence as a lower bound. We also use entropy and flow to describe the distribution of migration targets chosen by developers. They are both mature metrics on diversity analysis and directionality analysis in previous studies [51].

8.2.3 External Validity. It concerns whether we can generalize the results outside the scope of our study [38]. Firstly, our analysis depends on migration rules identified by commit messages explicitly mentioning migrations. Migration rules not explicitly mentioned in commit messages might be overlooked, potentially leading to an incomplete representation of migration activities. However, we believe the automated identification of self-admitted library migrations tends to have higher accuracy, and can capture less frequent migration patterns compared to those frequency-based methods. SALMs' commit messages also contain richer information, providing deeper insights into the process and rationale behind such changes. The rule-matching approach allows us to detect and analyze migrations that follow the same rules even when they are not explicitly mentioned in the commit messages, thereby providing a more comprehensive view of migration practices. More relevant information could be considered automatically, such as the identification of modified code, to provide a more precise and comprehensive result. Besides, the match of the commit message pattern only uses regular NLP techniques. The adoption of Large Language Models can also help capture the hidden descriptions of migration in commit messages. Secondly, our findings may not be generalizable to projects that use other package managers, build systems (e.g., CMake), or code clones. However, when parsing dependencies, we find that many projects use both CMake and other PMTs (vcpkg, conan) simultaneously to manage their dependency and explicitly declare the use of relevant PMTs in CMake. This suggests that we do not really omit too many migrations. Besides, we identified 2,191 migrations related to 538 libraries, which is a relatively large dataset for library migration [22]. The consideration of CMake may not introduce a significant influence on the results. Lastly, CMake scripts always involve intricate and dynamic scripting, making it difficult to reliably extract dependency information. The parsing of build scripts like CMake is error-prone and difficult, and we hope for more research in the future to enrich the results. Code cloning is also hard to be effectively and accurately detected. We choose as many dependency management methods as possible to mitigate this threat, including package managers and building systems, and analyze more than 120k blobs with dependency changes. And the way developers manage their dependencies should not significantly influence the characteristics of migrations we have found. Finally, the generalizability of our results to programming languages other than C/C++ is limited. For different languages, different migration characteristics may occur due to distinct application domains and development practices. Therefore, we compare the migration patterns in C/C++ with other programming languages [20, 22]. Both the divergences and similarities in how migrations occurred across different languages are revealed in our findings.

9 DATA AVAILABILITY

The replication package of our study is available at:

https://zenodo.org/records/17571861
https://github.com/guhaiqiao/C-CPP-Library-Migration

Our replication package is designed to support further research. It includes (1) a preprocessed, curated dataset of C/C++ library migrations, and (2) the accompanying Python scripts and Jupyter Notebooks necessary to reproduce the four research questions in our paper. We hope it serves as a valuable resource for future work in this area.

10 Conclusion

In this paper, we report a multidimensional comparative study of the prevalence, domain, target, and rationale of migrations in the C/C++ ecosystem. Our main contributions are:

1) A semi-automated mining method identifying migrations for C/C++ libraries based on git repositories.

2) A migration dataset for C/C++ libraries.

3) The nature of dependency management in the C/C++ ecosystem, especially compared to Python, JavaScript, and Java, including the overall trend of C/C++ library migrations, migrations among different package management tools, three frequent migration domains, i.e., GUI, Build, and OS development, one-to-one and unidirectional characteristics, and four C/C++-specific migration reasons.

Our study offers valuable insights for researchers and practitioners. For researchers, this work advances understanding of dependency management and the distinctive characteristics of library migrations in C/C++. The provided dataset supports deep analysis of migration behavior and the development and evaluation of automated migration tools. For practitioners, the migration domains and rationales that we identify provide actionable guidance for adopting or migrating libraries. Because one-to-one migrations are common in C/C++, the dataset can provide detailed examples when conducting migrations.

Our future work will focus on (i) integrating CMake-managed projects to enhance ecosystem coverage, and (ii) leveraging program analysis and large language models (LLMs) to achieve more comprehensive automatic library migration. We also encourage further research to broaden coverage and validate our findings, toward a more comprehensive understanding of C/C++ library migrations.

References

[1] Rabe Abdalkareem, Olivier Nourry, Sultan Wehaibi, Suhaib Mujahid, and Emad Shihab. 2017. Why do developers use trivial packages? an empirical case study on npm. In Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017, Paderborn, Germany, September 4-8, 2017, Eric Bodden, Wilhelm Schäfer, Arie van Deursen, and Andrea Zisman (Eds.). ACM, 385–395. doi:10.1145/3106237.3106267

[2] Aylton Almeida, Laerte Xavier, and Marco Túlio Valente. 2024. Automatic Library Migration Using Large Language Models: First Results. In Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement, ESEM 2024, Barcelona, Spain, October 24-25, 2024, Xavier Franch, Maya Daneva, Silverio Martínez-Fernández, and Luigi Quaranta (Eds.). ACM, 427–433. doi:10.1145/3674805.3690746

[3] Hussein Alrubaye, Deema Alshoaibi, Eman Abdullah AlOmar, Mohamed Wiem Mkaouer, and Ali Ouni. 2020. How Does Library Migration Impact Software Quality and Comprehension? An Empirical Study. In Reuse in Emerging Software Engineering Practices - 19th International Conference on Software and Systems Reuse, ICSR 2020, Hammamet, Tunisia, December 2-4, 2020, Proceedings (Lecture Notes in Computer Science), Sihem Ben Sassi, Stéphane Ducasse, and Hafedh Mili (Eds.). Springer, 245–260. doi:10.1007/978-3-030-64694-3_15

[4] Hussein Alrubaye, Mohamed Wiem Mkaouer, Igor Khokhlov, Leon Reznik, Ali Ouni, and Jason Mcgoff. 2020. Learning to recommend third-party library migration opportunities at the API level. Appl. Soft Comput. 90 (2020), 106140. doi:10.1016/J.ASOC.2020.106140

[5] Hussein Alrubaye, Mohamed Wiem Mkaouer, and Ali Ouni. 2019. MigrationMiner: An Automated Detection Tool of Third-Party Java Library Migration at the Method Level. In 2019 IEEE International Conference on Software Maintenance and Evolution, ICSME 2019, Cleveland, OH, USA, September 29 - October 4, 2019. IEEE, 414–417. doi:10.1109/ICSME.2019.00072

[6] Hussein Alrubaye, Mohamed Wiem Mkaouer, and Ali Ouni. 2019. On the use of information retrieval to automate the detection of third-party Java library migration at the method level. In Proceedings of the 27th International Conference on Program Comprehension, ICPC 2019, Montreal, QC, Canada, May 25-31, 2019, Yann-Gaël Guéhéneuc, Foutse Khomh, and Federica Sarro (Eds.). IEEE / ACM, 347–357. doi:10.1109/ICPC.2019.00053

[7] Lívia Barbosa and André C. Hora. 2022. How and Why Developers Migrate Python Tests. In IEEE International Conference on Software Analysis, Evolution and Reengineering, SANER 2022, Honolulu, HI, USA, March 15-18, 2022. IEEE, 538–548. doi:10.1109/SANER53432.2022.00071

[8] Thiago Tonelli Bartolomei, Krzysztof Czarnecki, and Ralf Lämmel. 2010. Swing to SWT and back: Patterns for API migration by wrapping. In 26th IEEE International Conference on Software Maintenance (ICSM 2010), September 12-18, 2010, Timisoara, Romania, Radu Marinescu, Michele Lanza, and Andrian Marcus (Eds.). IEEE Computer Society, 1–10. doi:10.1109/ICSM.2010.5610429

[9] TIOBE Software BV. 2024. TIOBE Index-TIOBE. https://www.tiobe.com/tiobe-index/.

[10] Chunyang Chen, Zhenchang Xing, Yang Liu, and Kent Ong Long Xiong. 2021. Mining Likely Analogical APIs Across Third-Party Libraries via Large-Scale Unsupervised API Semantics Embedding. IEEE Trans. Software Eng. 47, 3 (2021), 432–447. doi:10.1109/TSE.2019.2896123

[11] Bruce Collie, Philip Ginsbach, Jackson Woodruff, Ajitha Rajan, and Michael F. P. O'Boyle. 2020. M3: Semantic API Migrations. In 35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21-25, 2020. IEEE, 90–102. doi:10.1145/3324884.3416618

[12] Juri Di Rocco, Phuong T Nguyen, Claudio Di Sipio, Riccardo Rubei, Davide Di Ruscio, and Massimiliano Di Penta. 2025. DeepMig: A transformer-based approach to support coupled library and code migrations. Information and Software Technology 177 (2025), 107588.

[13] Mattia Fazzini, Qi Xin, and Alessandro Orso. 2020. APIMigrator: an API-usage migration tool for Android apps. In MOBILESoft '20: IEEE/ACM 7th International Conference on Mobile Software Engineering and Systems, Seoul, Republic of Korea, July 13-15, 2020, David Lo, Leonardo Mariani, and Ali Mesbah (Eds.). ACM, 77–80. doi:10.1145/3387905.3388608

[14] Inc Github. 2014. Example. https://github.com/cloudbotirc/cloudbot/commit/f82432.

[15] Inc Github. 2019. Example. https://github.com/Acidburn0zzz/doraskayo-mutter/commit/3693f6.

[16] Inc Github. 2021. Example. https://github.com/PazerOP/tf2_bot_detector/commit/d66045.

[17] Inc Github. 2022. Example. https://github.com/mo-xiaoming/pl0-jit/commit/f7e83b.

[18] Inc Github. 2022. Example. https://github.com/Keruspe/GPaste/commit/189bcc.

[19] Inc Github. 2022. Example. https://github.com/cfillion/reaimgui/commit/8e217b.

[20] Haiqiao Gu, Hao He, and Minghui Zhou. 2023. Self-Admitted Library Migrations in Java, JavaScript, and Python Packaging Ecosystems: A Comparative Study. In 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE, Taipa, Macao, 627–638. doi:10.1109/SANER56733.2023.00064

[21] Junxiao Han, Yarong Wang, Xiaodong Gu, Cuiyun Gao, Yao Wan, Song Han, David Lo, and Shuiguang Deng. 2025. LibRec: Benchmarking Retrieval-Augmented LLMs for Library Migration Recommendations. CoRR abs/2508.09791 (2025). arXiv:2508.09791 doi:10.48550/ARXIV.2508.09791

[22] Hao He, Runzhi He, Haiqiao Gu, and Minghui Zhou. 2021. A large-scale empirical study on Java library migrations: prevalence, trends, and rationales. In Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, Athens Greece, 478–490. doi:10.1145/3468264.3468571

[23] Hao He, Yulin Xu, Xiao Cheng, Guangtai Liang, and Minghui Zhou. 2021. MigrationAdvisor: Recommending Library Migrations from Large-Scale Open-Source Data. In 2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion). IEEE, Madrid, ES, 9–12. doi:10.1109/ICSE-Companion52605.2021.00023

[24] Hao He, Yulin Xu, Yixiao Ma, Yifei Xu, Guangtai Liang, and Minghui Zhou. 2021. A Multi-Metric Ranking Approach for Library Migration Recommendations. In 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). IEEE, Honolulu, HI, USA, 72–83. doi:10.1109/SANER50967.2021.00016

[25] Stack Exchange Inc. 2011. Questions. https://stackoverflow.com/questions/6692238.

[26] Mohayeminul Islam, Ajay Kumar Jha, Ildar Akhmetov, and Sarah Nadi. 2024. Characterizing Python Library Migrations. Proc. ACM Softw. Eng. 1, FSE, Article 5 (jul 2024), 23 pages. doi:10.1145/3643731

[27] Mohayeminul Islam, Ajay Kumar Jha, May Mahmoud, Ildar Akhmetov, and Sarah Nadi. 2025. Using LLMs for Library Migration. CoRR abs/2504.13272 (2025). arXiv:2504.13272 doi:10.48550/ARXIV.2504.13272

[28] Mohayeminul Islam, Ajay Kumar Jha, Sarah Nadi, and Ildar Akhmetov. 2023. PyMigBench: A Benchmark for Python Library Migration. In 20th IEEE/ACM International Conference on Mining Software Repositories, MSR 2023, Melbourne, Australia, May 15-16, 2023. IEEE, 511–515. doi:10.1109/MSR59073.2023.00075

[29] Suhas Kabinna, Cor-Paul Bezemer, Weiyi Shang, and Ahmed E. Hassan. 2016. Logging library migrations: a case study for the apache software foundation projects. In Proceedings of the 13th International Conference on Mining Software Repositories, MSR 2016, Austin, TX, USA, May 14-22, 2016, Miryung Kim, Romain Robbes, and Christian Bird (Eds.). ACM, 154–164. doi:10.1145/2901739.2901769

[30] Klaus Krippendorff. 2018. Content analysis: An introduction to its methodology. Sage publications.

[31] Raula Gaikovina Kula, Daniel M German, Ali Ouni, Takashi Ishio, and Katsuro Inoue. 2018. Do developers update their library dependencies? An empirical study on the impact of security advisories on library migration. Empirical Software Engineering 23 (2018), 384–417.

[32] Chengwei Liu, Sen Chen, Lingling Fan, Bihuan Chen, Yang Liu, and Xin Peng. 2022. Demystifying the Vulnerability Propagation and Its Evolution via Dependency Trees in the NPM Ecosystem. In 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022. ACM, 672–684. doi:10.1145/3510003.3510142

[33] Yuxing Ma, Chris Bogart, Sadika Amreen, Russell Zaretzki, and Audris Mockus. 2019. World of code: an infrastructure for mining the universe of open source VCS data. In Proceedings of the 16th International Conference on Mining Software Repositories, MSR 2019, 26-27 May 2019, Montreal, Canada, Margaret-Anne D. Storey, Bram Adams, and Sonia Haiduc (Eds.). IEEE / ACM, 143–154. doi:10.1109/MSR.2019.00031

[34] MediaWiki. 2024. Qt | Transition from Qt 4.x to Qt5. https://wiki.qt.io/Transition_from_Qt_4.x_to_Qt5.

[35] meson. 2024. meson | The meson build system. https://mesonbuild.com/.

[36] André Miranda and João Pimentel. 2018. On the use of package managers by the C++ open-source community. In Proceedings of the 33rd Annual ACM Symposium on Applied Computing, SAC 2018, Pau, France, April 09-13, 2018, Hisham M. Haddad, Roger L. Wainwright, and Richard Chbeir (Eds.). ACM, 1483–1491. doi:10.1145/3167132.3167290

[37] Suhaib Mujahid, Diego Elias Costa, Rabe Abdalkareem, and Emad Shihab. 2023. Where to Go Now? Finding Alternatives for Declining Packages in the npm Ecosystem. In 38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023. IEEE, 1628–1639. doi:10.1109/ASE56229.2023.00119

[38] Nasser Mustafa, Yvan Labiche, and Dave Towey. 2019. Mitigating Threats to Validity in Empirical Software Engineering: A Traceability Case Study. In 43rd IEEE Annual Computer Software and Applications Conference, COMPSAC 2019, Milwaukee, WI, USA, July 15-19, 2019, Volume 2, Vladimir Getov, Jean-Luc Gaudiot, Nariyoshi Yamai, Stelvio Cimato, J. Morris Chang, Yuuichi Teranishi, Ji-Jiang Yang, Hong Va Leong, Hossain Shahriar, Michiharu Takemoto, Dave Towey, Hiroki Takakura, Atilla Elçi, Susumu Takeuchi, and Satish Puri (Eds.). IEEE, 324–329. doi:10.1109/COMPSAC.2019.10227

[39] Ali Ouni, Raula Gaikovina Kula, Marouane Kessentini, Takashi Ishio, Daniel M German, and Katsuro Inoue. 2017. Search-based software library recommendation using multi-objective optimization. Information and Software Technology 83 (2017), 55–75.

[40] Amantia Pano, Daniel Graziotin, and Pekka Abrahamsson. 2018. Factors and actors leading to the adoption of a JavaScript framework. Empir. Softw. Eng. 23, 6 (2018), 3503–3534. doi:10.1007/S10664-018-9613-X

[41] Inc Reddit. 2019. Questions. https://www.reddit.com/r/cpp/comments/cmqawn/modern_c_json_serialization_library/.

[42] Inc Reddit. 2025. Questions. https://www.reddit.com/r/cpp/comments/1hbvzq4/any_good_c_ui_libraries/.

[43] Patrick Riehmann, Manfred Hanfler, and Bernd Froehlich. 2005. Interactive Sankey Diagrams. In IEEE Symposium on Information Visualization (InfoVis 2005), 23-25 October 2005, Minneapolis, MN, USA, John T. Stasko and Matthew O. Ward (Eds.). IEEE Computer Society, 233–240. doi:10.1109/INFVIS.2005.1532152

[44] Baihui Sang, Liang Wang, Jierui Zhang, and Xianping Tao. 2025. Attributed Multiplex Learning for Analogical Third-Party Library Recommendation and Retrieval. In 33rd IEEE/ACM International Conference on Program Comprehension, ICPC@ICSE 2025, Ottawa, ON, Canada, April 27-28, 2025. IEEE, 403–413. doi:10.1109/ICPC66645.2025.00051

[45] Wei Tang, Zhengzi Xu, Chengwei Liu, Jiahui Wu, Shouguo Yang, Yi Li, Ping Luo, and Yang Liu. 2022. Towards Understanding Third-party Library Dependency in C/C++ Ecosystem. In 37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10-14, 2022. ACM, 106:1–106:12. doi:10.1145/3551349.3560432

[46] Cédric Teyton, Jean-Rémy Falleri, and Xavier Blanc. 2012. Mining Library Migration Graphs. In 19th Working Conference on Reverse Engineering, WCRE 2012, Kingston, ON, Canada, October 15-18, 2012. IEEE Computer Society, 289–298. doi:10.1109/WCRE.2012.38

[47] Cédric Teyton, Jean-Rémy Falleri, Marc Palyart, and Xavier Blanc. 2014. A study of library migrations in java. Journal of Software: Evolution and Process 26, 11 (2014), 1030–1052.

[48] Yingchen Tian, Yuxia Zhang, Klaas-Jan Stol, Lin Jiang, and Hui Liu. 2022. What Makes a Good Commit Message?. In 44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27,

2022. ACM, 2389–2401. doi:10.1145/3510003.3510205

[49] Enrique Larios Vargas, Maurício Finavaro Aniche, Christoph Treude, Magiel Bruntink, and Georgios Gousios. 2020. Selecting third-party libraries: the practitioners' perspective. In ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 245–256. doi:10.1145/3368089.3409711

[50] Ying Wang, Bihuan Chen, Kaifeng Huang, Bowen Shi, Congying Xu, Xin Peng, Yijian Wu, and Yang Liu. 2020. An Empirical Study of Usages, Updates and Risks of Third-Party Libraries in Java Projects. In IEEE International Conference on Software Maintenance and Evolution, ICSME 2020, Adelaide, Australia, September 28 - October 2, 2020. IEEE, 35–45. doi:10.1109/ICSME46990.2020.00014

[51] Xiangchen Wu, Liang Wang, Zhiwen Zheng, Baihui Sang, Jierui Zhang, and Xianping Tao. 2025. An entropy-based measure of fork diversity and its correlations with open source software projects' received contributions. Empir. Softw. Eng. 30, 4 (2025), 111. doi:10.1007/S10664-025-10668-4

[52] Shengzhe Xu, Ziqi Dong, and Na Meng. 2019. Meditor: inference and application of API migration edits. In Proceedings of the 27th International Conference on Program Comprehension, ICPC 2019, Montreal, QC, Canada, May 25-31, 2019, Yann-Gaël Guéhéneuc, Foutse Khomh, and Federica Sarro (Eds.). IEEE / ACM, 335–346. doi:10.1109/ICPC.2019.00052

[53] Weiwei Xu, Hao He, Kai Gao, and Minghui Zhou. 2023. Understanding and Remediating Open-Source License Incompatibilities in the PyPI Ecosystem. In 38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023. IEEE, 178–190. doi:10.1109/ASE56229.2023.00175

[54] Likang Yin and Vladimir Filkov. 2020. Team Discussions and Dynamics During DevOps Tool Adoptions in OSS Projects. In 35th IEEE/ACM International Conference on Automated Software Engineering, ASE 2020, Melbourne, Australia, September 21-25, 2020. IEEE, 697–708. doi:10.1145/3324884.3416640

[55] Rodrigo Elizalde Zapata, Raula Gaikovina Kula, Bodin Chinthanet, Takashi Ishio, Kenichi Matsumoto, and Akinori Ihara. 2018. Towards Smoother Library Migrations: A Look at Vulnerable Dependency Migrations at Function Level for npm JavaScript Packages. In 2018 IEEE International Conference on Software Maintenance and Evolution, ICSME 2018, Madrid, Spain, September 23-29, 2018. IEEE Computer Society, 559–563. doi:10.1109/ICSME.2018.00067

[56] Ahmed Zerouali and Tom Mens. 2017. Analyzing the evolution of testing library usage in open source Java projects. In IEEE 24th International Conference on Software Analysis, Evolution and Reengineering, SANER 2017, Klagenfurt, Austria, February 20-24, 2017, Martin Pinzger, Gabriele Bavota, and Andrian Marcus (Eds.). IEEE Computer Society, 417–421. doi:10.1109/SANER.2017.7884645

[57] Ahmed Zerouali, Tom Mens, Alexandre Decan, and Coen De Roover. 2022. On the impact of security vulnerabilities in the npm and RubyGems dependency networks. Empir. Softw. Eng. 27, 5 (2022), 107. doi:10.1007/S10664-022-10154-1